Welcome to Colab!

```
!pip install -q ultralytics opencv-python-headless moviepy
```

```
# IMPORTS
import cv2
import numpy as np
from ultralytics import YOLO
from collections import defaultdict, deque
from moviepy.editor import VideoFileClip, AudioClip
# SETTINGS
VIDEO_PATH = "/content/video.mp4"
OUTPUT_PATH = "/content/output.mp4"
FINAL_OUTPUT = "/content/final_output.mp4"

PIXEL_TO_METER = 0.04
MIN_ACCIDENT_SPEED = 8
IOU_THRESHOLD = 0.3
# LOAD MODEL
model = YOLO("yolov8n.pt")
# VIDEO SETUP
cap = cv2.VideoCapture(VIDEO_PATH)

if not cap.isOpened():
    raise Exception("Video not found")

fps = cap.get(cv2.CAP_PROP_FPS)
if fps == 0:
    fps = 30

w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

out = cv2.VideoWriter(
    OUTPUT_PATH,
    cv2.VideoWriter_fourcc(*'mp4v'),
    fps,
    (w, h)
)
# STORAGE
```

```
trajectories = defaultdict(lambda: deque(maxlen=10))
speed_history = defaultdict(lambda: deque(maxlen=5))
seen_ids = set()
alerted_ids = set()
accident_ids = set()

alert_times = []
frame_count = 0

# FUNCTIONS
def compute_speed(p1, p2):
    dist = np.linalg.norm(np.array(p1) - np.array(p2))
    return dist * PIXEL_TO_METER * fps * 3.6

def compute_iou(a, b):
    xA = max(a[0], b[0])
    yA = max(a[1], b[1])
    xB = min(a[2], b[2])
    yB = min(a[3], b[3])
    inter = max(0, xB-xA)*max(0, yB-yA)
    if inter == 0:
        return 0
    areaA = (a[2]-a[0])*(a[3]-a[1])
    areaB = (b[2]-b[0])*(b[3]-b[1])
    return inter/(areaA+areaB-inter)

def wrong_direction(traj):
    if len(traj) < 5:
        return False
    dx = traj[-1][0] - traj[0][0]
    return dx < -30   # reverse motion

# MAIN PROCESSING
results = model.track(
    source=VIDEO_PATH,
    persist=True,
    tracker="bytetrack.yaml",
    stream=True,
    verbose=False
)

for result in results:

    frame_count += 1
    time_sec = frame_count / fps
```

```
        frame = result.orig_img.copy()

        if result.boxes is not None and result.boxes.id is not None:

            boxes = result.boxes.xyxy.cpu().numpy()
            ids = result.boxes.id.cpu().numpy().astype(int)
            classes = result.boxes.cls.cpu().numpy().astype(int)

            current_boxes = {}

            for box, track_id, cls in zip(boxes, ids, classes):

                if cls not in [2,3,5,7]:
                    continue

                x1,y1,x2,y2 = map(int, box)
                cx, cy = (x1+x2)//2, (y1+y2)//2

                trajectories[track_id].append((cx,cy))
                current_boxes[track_id] = (x1,y1,x2,y2)

                # Vehicle entry alert
                if track_id not in seen_ids:
                    seen_ids.add(track_id)
                    alert_times.append(time_sec)

                # Wrong direction alert
                if wrong_direction(trajectories[track_id]):
                    if track_id not in alerted_ids:
                        alerted_ids.add(track_id)
                        alert_times.append(time_sec)

                # Accident (sudden stop)
                if len(trajectories[track_id]) > 1:
                    speed = compute_speed(
                        trajectories[track_id][-2],
                        trajectories[track_id][-1]
                    )

                    speed_history[track_id].append(speed)

                    if speed < MIN_ACCIDENT_SPEED and np.mean(speed_history[track_id]) > 20:
                        if track_id not in accident_ids:
                            accident_ids.add(track_id)
                            alert_times.append(time_sec)
```

```
                    alert_times.append(time_sec)

        # Collision detection
        ids_list = list(current_boxes.keys())
        for i in range(len(ids_list)):
            for j in range(i+1, len(ids_list)):
                id1,id2 = ids_list[i], ids_list[j]

                if compute_iou(current_boxes[id1], current_boxes[id2]) > IOU_THRESHOLD:
                    for idx in [id1,id2]:
                        if idx not in accident_ids:
                            accident_ids.add(idx)
                            alert_times.append(time_sec)

        # DRAW
        for track_id, box in current_boxes.items():
            x1,y1,x2,y2 = box

            color = (0,255,0)
            label = f"ID {track_id}"

            if track_id in accident_ids:
                color = (0,0,255)
                label += " ACCIDENT"
            elif track_id in alerted_ids:
                color = (0,255,255)
                label += " WRONG WAY"

            cv2.rectangle(frame,(x1,y1),(x2,y2),color,2)
            cv2.putText(frame,label,(x1,y1-10),
                        cv2.FONT_HERSHEY_SIMPLEX,0.5,color,2)

    out.write(frame)

cap.release()
out.release()

print("Video processing done")

# 🔊 ADD SOUND TO VIDEO
video = VideoFileClip(OUTPUT_PATH)

def make_beep(t):
    # Ensure t is always a NumPy array of floats
    t_processed = np.atleast_1d(t).astype(float)
    beep_signal = np.zeros_like(t_processed)
```

```
    beep_signal = np.zeros_like(t_processed)

    for at in alert_times:
        mask = np.abs(t_processed - at) < 0.05 # Adjusted to 0.05 for shorter beep
        beep_signal[mask] += np.sin(2 * np.pi * 1000 * t_processed[mask])

    return beep_signal

audio = AudioClip(make_beep, duration=video.duration, fps=44100)

final = video.set_audio(audio)
final.write_videofile(FINAL_OUTPUT)

print("Final video with sound ready")
```

```
Video processing done
Moviepy - Building video /content/final_output.mp4.
MoviePy - Writing audio in final_outputTEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/final_output.mp4

Moviepy - Done !
Moviepy - video ready /content/final_output.mp4
Final video with sound ready
```

```
import io
import base64
from IPython.display import HTML

def show_video(file_path):
    video_file = io.open(file_path, 'r+b').read()
    encoded = base64.b64encode(video_file)

    return HTML(data=f'''
        <video width="640" height="480" controls>
            <source src="data:video/mp4;base64,{encoded.decode('ascii')}" type="video/mp4" />
        </video>
    ''')

# Use it like this:
show_video('/content/final_output.mp4')
```

0:14 / 0:14